full-mhs-z: implementation

Technical notes on `run.mode = "full-mhs-z"` (the M5 joint Gauss–Newton driver, M5i defaults). HSEMHS, 24 Oct 2026.

1. Running it

```
[mpiexec -n R] julia --project=. -t T scripts/run.jl config/modes/full-mhs-z.toml
```

`scripts/run.jl` validates the config, exports the M5 switches as environment variables (`MHS_INPUT`, `MHS_EOS_TABLE`, `MHS_SIGMA_REL`, `MHS_TRUST`, `MHS_STOP`, `MHS_PHI_RTOL`; they are read as constants when the driver is included), includes `scripts/m5_bifrost/m5h_fullres.jl` and calls `main([output, run, wall, ckpt, init_iter, restart])`. The driver can still be run directly with the same arguments.

Key	Default (M5i)	Meaning
`input.file`	required	height-grid cube (production `IO/input-mhs-hse-fullz.nc`, $504 \times 504 \times 496$)
`run.output`	required	run directory
`full_mhs.run`	`run`	`run`: native grid; `smoke`: 42×42, every second level, every code path in minutes
`full_mhs.wall_hours`, `ckpt_hours`	24, 1	wall cap; checkpoint interval
`full_mhs.init_iter`	3000	CG iterations of the field initialisation
`full_mhs.restart`	`""`	any saved file (`init.h5`, `atmosphere_A/B.h5`, final)
`full_mhs.sigma_rel`	0	$\sigma_B = \sigma_{\rm rel}\lvert\mathbf{B}_{\rm obs}\rvert \oplus 5\,\mathrm{G}$ (M5h: 0.05)
`full_mhs.trust_region`	false	false: line search; true: trust region (M5h)
`full_mhs.stop`, `phi_rtol`	`phi`, 10^{-6}	$\lvert\Delta\Phi\rvert/\Phi <$ rtol on 3 consecutive accepted steps; `dz`: $\max\lvert\Delta z_{\tau 1}\rvert < 1\,\mathrm{mm}$

2. Problem setup (`bif_setup.jl`, `bif_problem`)

Reads T, $\mathbf{B}_{\rm obs}$, coordinates and the attributes `P_ref_Pa`, `period_x_m`, `period_y_m`, `gravity_m_s2`; `init_parallel` (2-D blocks); periodic `Grid`; σ per component from `sigma_synth`; `SMHSProblem` with $\lambda_F = 10^{-4}$, $\lambda_Q = 10^{-3}$, $\lambda_S = 10^{-3}$, $\lambda_U = 0$, data on faces. State $x = (v, \tilde{A}_x, \tilde{A}_y, \tilde{A}_z, \bar{b})$: $u = v - \langle v\rangle_{\rm bottom} + \ln P_{\rm ref}$ at centres (the gauge as a parameterisation), $\mathbf{A} = A_{\rm ref}\tilde{\mathbf{A}}$ on edges ($A_{\rm ref}$ = median σ × mean spacing), $\mathbf{B} = \nabla\times\mathbf{A} + \bar{b}\hat{\mathbf{z}}$ on faces, $\bar{b}$ a scalar stored on rank 0.

- `init_state`: `staggered_hse!`, the exact discrete vertical balance $(P_k - P_{k+1})/\Delta z_{k+1/2} + g\sqrt{\rho_k\rho_{k+1}} = 0$ upward from $P_{\rm ref}$ (Brent per level); $\bar{b} = \langle B_{z,\rm obs}\rangle$.
- `set_scales!`: $S = \max(\rho_0 g,\ 0.01\,\mathrm{median}_k\,\rho_0 g)$ at centres, $S_f^2 = S_a S_b$ on faces; Φ_Q layer weights $(B_{\rm ref}/S)^2$ from layer medians. Fixed for the solve.
- `init_A!`: $(\tilde{\mathbf{A}}, \bar{b})$ fitted to $\mathbf{B}_{\rm obs}$ with v frozen ($\Phi_B + \Phi_S$ only, quadratic): CG on the normal equations with $Qp = \nabla\Phi(p) - \nabla\Phi(0)$; converges to the minimum-norm solution in the curl gauge null space.
- Pressure floor `ulo` $= u_0 - 3$.

3. Objective (`MHSStaggered.jl`, `objective!`, `gradient!`)

$\Phi = \Phi_F + \Phi_B + \Phi_S + \Phi_Q$ (+ optional Φ_U, Φ_N, off): $\Phi_F = (\lambda_F/N_F)\sum_f w_f |R_f|^2/S_f^2$ on x- and y-faces of layers $2\ldots N_z - 1$ and the interior z-faces; $\Phi_B = (1/N_B)\sum((B_f - B_{{\rm obs},f})/\sigma_f)^2$ on faces ($\mathbf{B}_{\rm obs}$ and $1/\sigma^2$ interpolated to faces); $\Phi_S = (\lambda_S/N)\sum w \sum_a (h_a^2\partial_a^2\delta B)^2/\sigma^2$ at centres; $\Phi_Q = (\lambda_F\lambda_Q/N_F)\sum_e w_e \sum_a (B_{\rm ref}\tfrac{1}{4}\delta_a^2 J_e/S_e)^2$ on edges. On an x-face the force uses $\partial_x P$ from the two centres, J_y, J_z averaged from the nearest edges, B_y, B_z from the four nearest faces; on z-faces $\rho = \sqrt{\rho_k\rho_{k+1}}$. The gradient is a hand-written adjoint of every stencil, including the EOS term $\partial\ln\rho/\partial\ln P|_T$ from the table.

4. Solver (`MHSJointGN.jl`, `joint_gn_solve!`)

One quadratic model per iteration at the accepted base state:

1. PCG (Steihaug–Toint) on $(H_{\rm GN} + \mu D)\,y = -g$, $H_{\rm GN}$ products by `gn_hvp!` (linearised forward plus adjoint), D the preconditioner diagonal (`gn_diagonal!`: exact GN diagonal for v, per-layer Hutchinson estimates for $\tilde{\mathbf{A}}$), multiplied by `block_scale` $= (100, 1, 1)$ for $(v, \tilde{\mathbf{A}}, \bar{b})$; pressure nodes at the floor with an outward gradient fixed; at most 100 iterations, relative residual 10^{-2}.
2. Candidate: the pressure part re-centred on the anchor and projected onto the box $\max(u_{\rm lo} - u, -5) \le \delta u \le 5$; the field part unconstrained.

3. Projected-Cauchy safeguard along $-D^{-1}g$ (exact model minimiser without a trust region); used if the GN step achieves less than 0.1 of the Cauchy model decrease.
4. Trial in a separate workspace. M5i (`trust_region = false`): accept if Φ decreases (actual/predicted $> 10^{-4}$, floor respected), otherwise halve along the same direction (up to 30 times); μ fixed at 10^{-6} (gauge null space only). M5h: trust radius in the D-norm, μ and Δ adapted from the ratio.

The curl-free (gauge) directions of $\tilde{\mathbf{A}}$ carry no curvature; μD keeps the system definite and gauge components change neither $\mathbf{B}$ nor Φ.

5. Driver (`m5h_fullres.jl`)

After every accepted step (callback): $z(\tau_{500}{=}1)$ in every column from the current P (`tau1_heights!`, `exp_tail`), a row in `history.csv` (Φ and its terms, $dz_{\max}$, force and data statistics, step data), the stop test, checkpoints every `ckpt_hours` alternating `atmosphere_A/B.h5` (written to `.tmp`, then renamed), a `STOP` file check and the wall cap. Every candidate goes to `candidates.csv`. Saved solver arrays are global owned values (no halos), so a restart may use another rank count; on restart the $z(\tau{=}1)$ map is recomputed and checked against the saved one (flags exact, heights to 10^{-6} m). Any error writes `atmosphere_crash.h5` (single rank) or aborts all ranks.

6. Parallelisation

2-D (x, y) blocks with one halo layer (`exchange_halos!`), `THREAD_FUNNELED`, threads inside each block. Global sums use a fixed reduction order (row partials, `reduce_rows`), so gradients and objectives are reproducible; tests check thread and rank invariance of the staggered gradient. Output is cooperative (rank 0 creates the file, then each rank writes its hyperslab in turn); no parallel HDF5. Multi-node: MPICH via `mpiexecjl` with the ssh launcher, each rank wrapped in `mhs_rank.sh` (libfabric TCP on the IPoIB interface, node-local Julia), see `config/modes/README.md`.

7. Verification and performance

- Test suite: staggered gradient against finite differences (ideal gas, Wittmann table), forward/adjoint allocations, thread and rank invariance, the exact discrete HSE initialiser, Gate C1b (uniform $\mathbf{B}$), C2 (potential field), C3 (checkerboard modes), and the M5–M5g solver tests (projection, block termination, trust ball, barrier and face bounds).
- Through `run.jl`: smoke run on 4 ranks ($42 \times 42 \times 249$) completed 3 accepted steps, checkpoints and the final save; the log confirmed $\sigma_B = 5$ G uniform, no trust region, Φ stop at 10^{-6}.
- Production M5i (22–23 Oct): 24 ranks $\times$ 16 threads on 4 nodes, $\approx 5 \times 10^8$ unknowns; 112 accepted steps, $\Phi = 7.88 \times 10^3$ when stopped on request. M5h (trust region, 5 % σ) ran to its 24 h wall cap.